

Exercise – Understanding hypothesis testing using simulation

1. Aim of the exercise

The aim of this exercise is to explain hypothesis testing using the t test through simulation. Without simulation, the logic underlying hypothesis testing can be difficult to grasp, which is why students often end up memorizing the steps of the procedure rather than developing an intuitive understanding of how it works.

2. Define the t value and the degrees of freedom

To illustrate the logic of hypothesis testing, assume that using some sample data, the number of degrees of freedom is 20 and the test statistic yields

$$t = \frac{\hat{\beta}_k - \beta_k^0}{SE(\hat{\beta}_k)} = 2.$$

We would like to determine whether this test statistic provides sufficient evidence against the null hypothesis. Specifically, we wish to test the null hypothesis

$$H_0 : \beta_k \leq \beta_k^0$$

against the alternative hypothesis

$$H_1 : \beta_k > \beta_k^0.$$

This is a right-tailed test for a regression coefficient.

```
8 %% 2. Define t value and degrees of freedom
9
10 % 2.1. Define t value
11 tValue = 2;
12
13 % 2.2. Define degrees of freedom
14 nu = 20;
```

3. Generate Student's t distributed values

We begin by generating a large number of random draws, in particular 20,000, from the Student's t distribution with 20 degrees of freedom and plotting the resulting frequency distribution. The draws are generated using the stochastic representation introduced in earlier exercises on the t -distribution.

It is important to keep in mind that these values are entirely hypothetical. They are not obtained from real data, nor do they correspond to the outcome of an actual experiment. Instead, each draw represents a possible value of the test statistic that could occur purely by random sampling variation when the null hypothesis is true.

By generating thousands of such draws, we obtain an approximation to the sampling distribution of the test statistic under the assumption that the null hypothesis is true, commonly referred to as the “null distribution”.

The null distribution represents our working description of reality. It shows what outcomes we would typically expect to observe if that description were correct. We will use the single observation available to us, namely the t -statistic computed from our sample, to assess whether this description provides a plausible representation of the true data-generating process.

```

16 %% 3. Generate Student's t distributed values
17
18 % 3.1. Number of random draws
19 N = 20000;
20
21 % 3.2. Define histogram bin edges
22 binEdges = -6:0.01:6;
23
24 % 3.3. Construct t-distributed random variables
25 draws = StochasticRepresentation(N,nu);
26
27 % 3.4. Create histogram
28 figure;
29 set(gcf,'Position',[100 100 1000 1000]);
30 histogram(draws,...
31     'BinEdges',binEdges,...
32     'FaceColor',[0.700 0.700 0.700],...
33     'EdgeAlpha',0,...
34     'DisplayName','All simulated values');
35 xmin = icdf('T',0.0001,nu);
36 xmax = icdf('T',0.9999,nu);
37 xlim([xmin xmax]);
38 title(['Fig. 1. Histogram of simulated t-distributed draws', ...
39     ', N = ', num2str(N), ...
40     ', \nu = ', num2str(nu)]);
41 xlabel('t');
42 ylabel('Frequency');
43 legend('show');

```

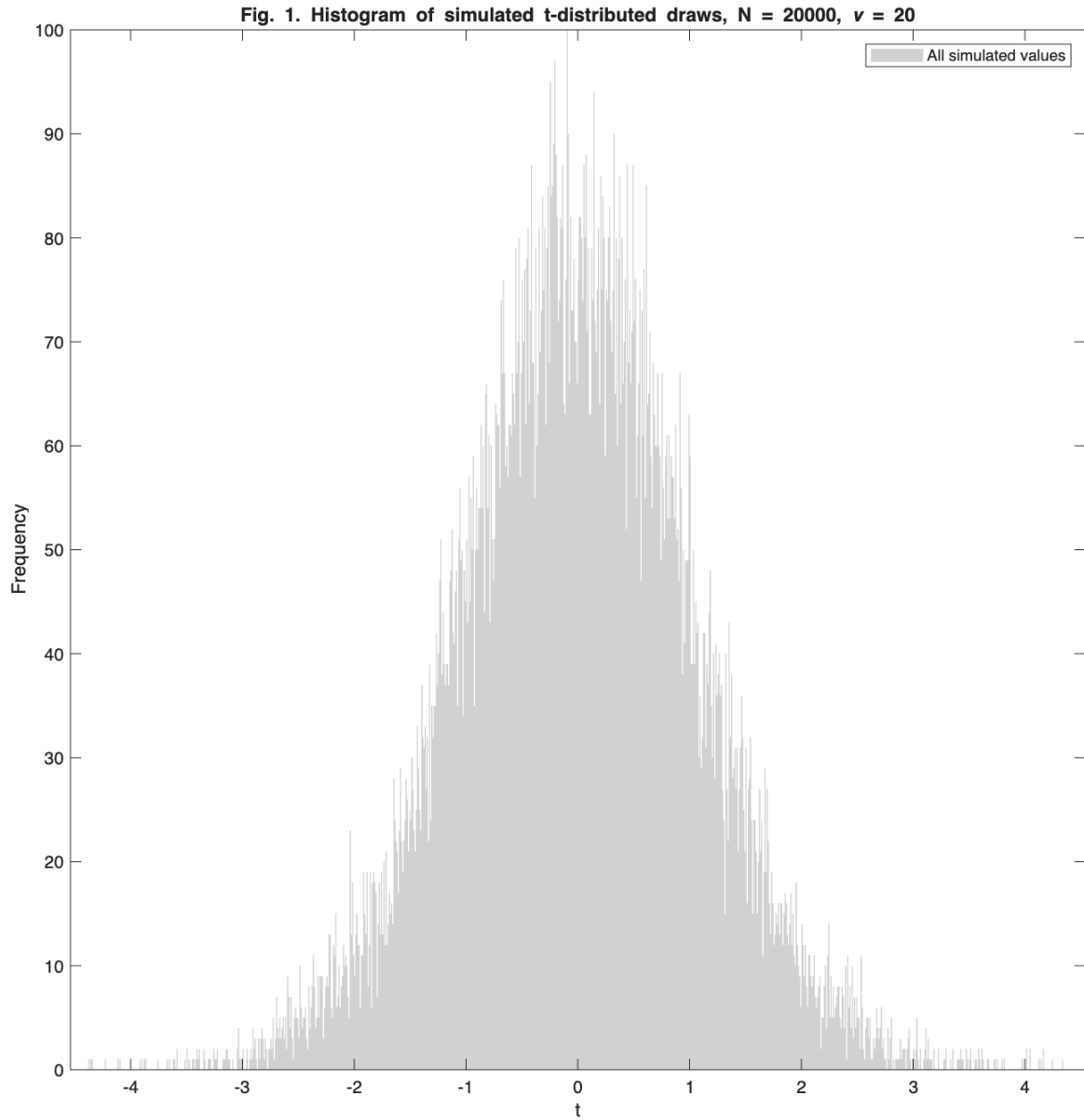


Figure 1: Approximation to the null distribution obtained from 20,000 simulated draws from a Student's t distribution with 20 degrees of freedom

4. Add the test statistic to the plot

In the frequency distribution of the simulated draws, we mark the observed test statistic from our example, which is 2. The vertical line indicates this value.

```
45 %% 4. Add the test statistic to the plot
46
47 % Plot histogram and test statistic
48 figure;
49 set(gcf,'Position',[100 100 1000 1000]);
50 histogram(draws,...
51     'BinEdges',binEdges,...
52     'FaceColor',[0.700 0.700 0.700],...
53     'EdgeAlpha',0,...
54     'DisplayName','All simulated values');
55 hold on
56 xline(tValue,...
57     'LineStyle',':',...
58     'Color',[0.541 0.133 0.702],...
59     'LineWidth',1.4,...
60     'DisplayName','Test statistic');
61 xmin = icdf('T',0.0001,nu);
62 xmax = icdf('T',0.9999,nu);
63 xlim([xmin xmax]);
64 hold off
65 title(['Fig. 2. Histogram of simulated t-distributed draws', ...
66     ', N = ',num2str(N), ...
67     ', \nu = ',num2str(nu)]);
68 xlabel('t');
69 ylabel('Frequency');
70 legend('show');
```

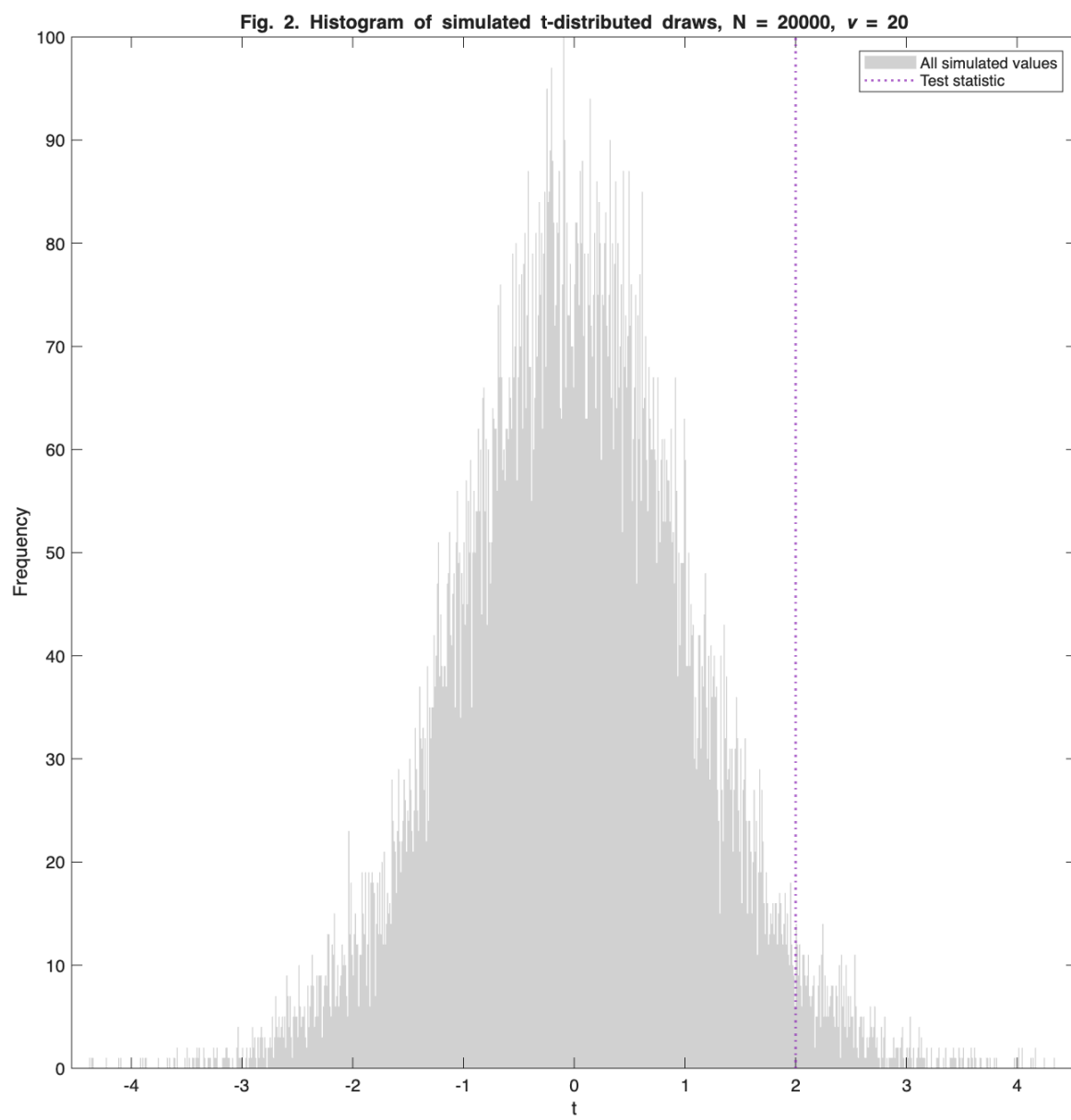


Figure 2: Approximation to the null distribution and observed test statistic

5. Highlight the extreme values and estimate the p -value by simulation

We define a variable named `extremeValues` containing all simulated draws from the null distribution that exceed the observed test statistic, 2. These draws are highlighted in the frequency distribution.

The variable `simulatedPValue` computes the proportion of simulated draws that are at least as extreme as the observed test statistic. We obtain a value of 0.0312, indicating that approximately 3.12% of the simulated draws exceed the observed value. This simulated proportion provides an estimate of the right-tail probability, or equivalently, the one-sided p -value. Because this estimate is based on random simulation, its exact value will vary slightly from run to run. Increasing the number of simulated draws reduces this simulation error and yields a more precise estimate of the true p -value.

```
72 %% 5. Highlight the extreme values and estimate the p-value by simulation
73
74 % 5.1. Select values exceeding the test statistic
75 extremeValues = draws(draws > tValue);
76
77 % 5.2. Plot histogram and highlight values exceeding the test statistic
78 figure;
79 set(gcf,'Position',[100 100 1000 1000]);
80 histogram(draws,...
81     'BinEdges',binEdges,...
82     'FaceColor',[0.700 0.700 0.700],...
83     'EdgeAlpha',0,...
84     'DisplayName','All simulated values');
85 hold on
86 histogram(extremeValues,...
87     'BinEdges',binEdges,...
88     'FaceColor',[0.541 0.133 0.702],...
89     'EdgeAlpha',0,...
90     'DisplayName','Values exceeding the test statistic');
91 xline(tValue,...
92     'LineStyle',':',...
93     'Color',[0.541 0.133 0.702],...
94     'LineWidth',1.4,...
95     'DisplayName','Test statistic');
96 xmin = icdf('T',0.0001,nu);
97 xmax = icdf('T',0.9999,nu);
98 xlim([xmin xmax]);
99 hold off
100 title(['Fig. 3. Simulated values exceeding the test statistic', ...
101     ', N = ',num2str(N), ...
102     ', \nu = ',num2str(nu)]);
103 xlabel('t');
104 ylabel('Frequency');
105 legend('show');
106
107 % 5.3. Estimate the right-tail probability (simulated p-value)
108 simulatedPValue = numel(extremeValues)/numel(draws);
```

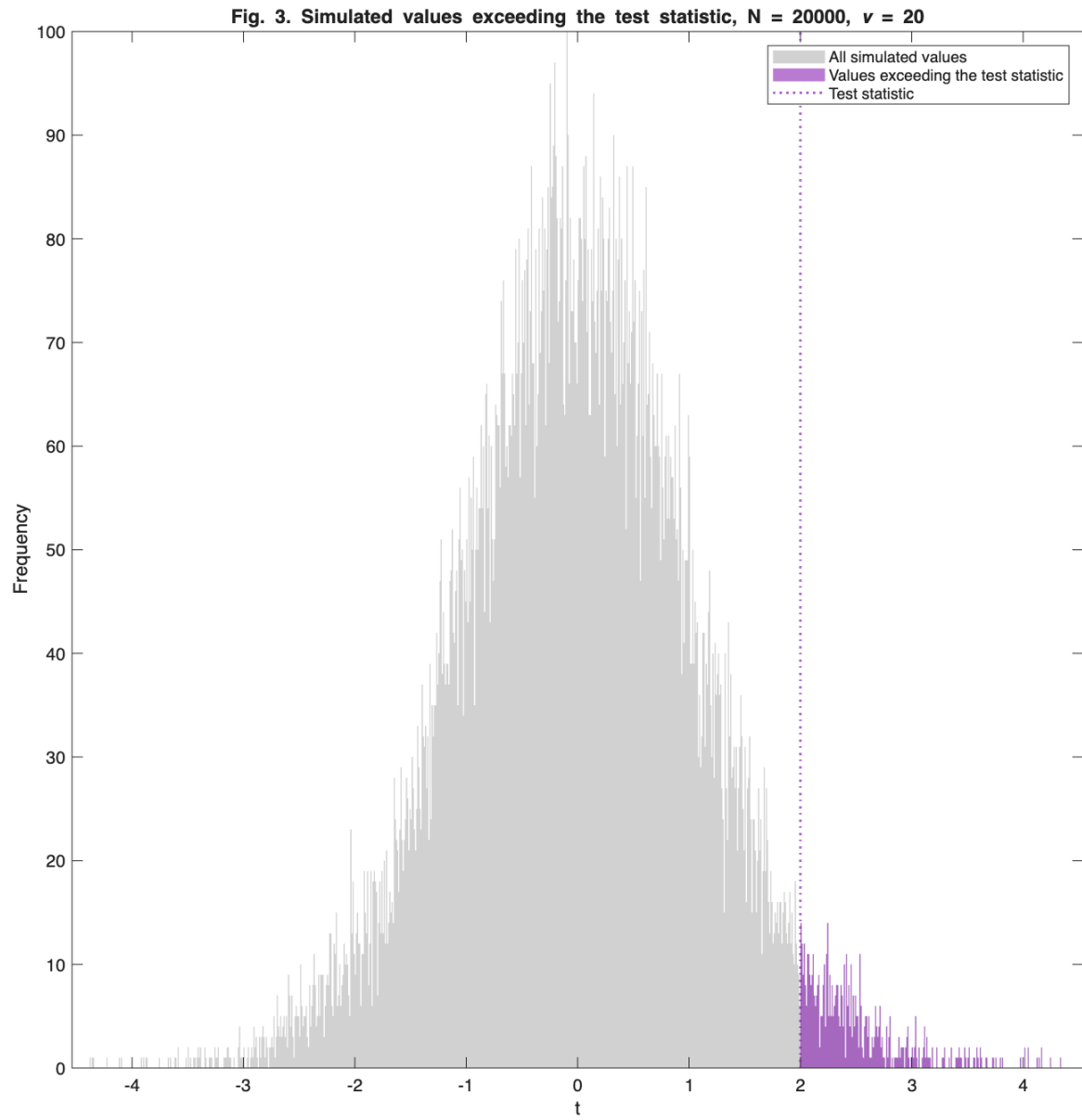


Figure 3: Approximation to the null distribution with the rejection region highlighted

6. The hypothesis test

The aim is to determine whether the observed test statistic provides sufficient evidence against the null hypothesis as represented by the null distribution.

There are two ways to conduct the test. In the first, we compare the observed test statistic with an appropriate threshold value. For a right-tailed test at the 5% significance level, this threshold value is

$$t_{0.95,20} = 1.725.$$

The notation $t_{0.95,20}$ denotes the 95th percentile of the t -distribution with 20 degrees of freedom. Consequently, 95% of the distribution lies to the left of 1.725 and 5% lies to the right.

Because

$$2 > 1.725,$$

the observed test statistic exceeds the threshold value and falls in the rejection region. We therefore reject the null hypothesis

$$H_0 : \beta_k \leq \beta_k^0$$

in favor of the alternative

$$H_1 : \beta_k > \beta_k^0.$$

Equivalently, the estimated coefficient lies 2 estimated standard errors above the hypothesized value β_k^0 :

$$\hat{\beta}_k - \beta_k^0 = 2 \text{SE}(\hat{\beta}_k).$$

Since

$$2 \text{SE}(\hat{\beta}_k) > 1.725 \text{SE}(\hat{\beta}_k),$$

the estimated coefficient lies farther above the hypothesized value than the threshold requires, leading to the same conclusion. This illustrates how the comparison with the null distribution is carried out. The t -statistic measures how far the estimate, $\hat{\beta}_k$, lies from the centre of the null distribution, β_k^0 , in units of standard errors, allowing us to assess whether the observed discrepancy is large relative to the uncertainty predicted by the null hypothesis.

The mechanics of the test are straightforward. The underlying rationale, however, is sometimes a source of confusion.

Suppose that the observed test statistic lies far in the tail of the null distribution. A common source of confusion stems from the existence of two possibilities. First, the null distribution is a correct working description of reality and we simply happened to observe a rare realization. Second, the null distribution is an incorrect working description of reality and we observed a value that is not surprising under the true distribution. This confusion is valid because we do not observe the true data-generating distribution but instead work with the hypothetical null distribution. From a single realization of the test statistic at hand, we can never be certain about either of the two possibilities.

Hypothesis testing resolves this ambiguity by building the testing procedure entirely on the first possibility. The null distribution is treated as a working model of reality, and the test asks whether the observed statistic looks typical relative to that model. If the observed value falls sufficiently far into the tail of the null distribution, we interpret this as evidence against the null hypothesis.

In contrast, the second possibility is never used to construct a competing reference distribution. This is why we state that we reject the null hypothesis rather than accept the alternative

hypothesis. Accepting the alternative would require knowing the exact true distribution, which remains unknown because we observe only a single sample. We simply compare our single realization against the hypothetical null distribution and evaluate its plausibility.

We can formalize this distinction using conditional probabilities. The first possibility corresponds to the probability of observing data such as ours under the assumption that the null hypothesis is true,

$$\mathbb{P}(\text{data} \mid H_0).$$

This is the quantity underlying the null distribution and the p -value. It asks whether the observed test statistic would be unusual if the null hypothesis were the correct description of reality. The second possibility shifts the conditioning in the opposite direction and concerns the probability that the null hypothesis is true after observing the data,

$$\mathbb{P}(H_0 \mid \text{data}).$$

This is sometimes the quantity that students and applied researchers have in mind when interpreting a hypothesis test. However, classical hypothesis testing does not provide this probability. Instead, it evaluates the plausibility of the observed data under the assumption that the null hypothesis is true. Much of the confusion surrounding p -values arises because these two conditional probabilities are fundamentally different, yet they are often conflated in interpretation.

Since

$$3.12\% < 5.00\%,$$

the simulated p -value is smaller than the chosen significance level. We therefore reject the null hypothesis. This is the same conclusion obtained earlier using the critical-value approach.

```

110 %% 6. The critical-value and p-value approaches to hypothesis testing
111
112 % 6.1. Define the critical value and rejection region
113 criticalValue = 1.725;
114 criticalValueRegion = draws(draws > 1.725);
115 pValue = mean(draws > tValue);
116 pValueRegion = draws(draws > tValue);
117
118 % 6.2. Add the decision rule to the histogram
119 figure;
120 set(gcf,'Position',[100 100 1000 1000]);
121 % Full distribution
122 histogram(draws,...
123     'BinEdges',binEdges,...
124     'FaceColor',[0.700 0.700 0.700],...
125     'EdgeAlpha',0,...
126     'DisplayName','All simulated values');
127 hold on
128 % Rejection region
129 histogram(criticalValueRegion,...
130     'BinEdges',binEdges,...
131     'FaceColor',[0.604 0.604 0.988],...
132     'FaceAlpha',0.15,...
133     'EdgeAlpha',0,...

```

```

134     'DisplayName','Critical value region');
135 % P-value region
136 histogram(pValueRegion,...
137     'BinEdges',binEdges,...
138     'FaceColor',[0.541 0.133 0.702],...
139     'FaceAlpha',1,...
140     'EdgeAlpha',0,...
141     'DisplayName','p-value region');
142 % Critical value
143 xline(1.725,...
144     'LineStyle',':',...
145     'Color',[0.604 0.604 0.988],...
146     'LineWidth',1.4,...
147     'DisplayName','Critical value');
148 % Test statistic
149 xline(tValue,...
150     'LineStyle',':',...
151     'Color',[0.541 0.133 0.702],...
152     'LineWidth',1.4,...
153     'DisplayName','Test statistic');
154 xmin = icdf('T',0.0001,nu);
155 xmax = icdf('T',0.9999,nu);
156 xlim([xmin xmax]);
157 hold off
158 title(['Fig. 4. Simulated values exceeding the test statistic',...
159     ', N = ',num2str(N), ...
160     ', \nu = ',num2str(nu)...
161     ', t = ',num2str(tValue,'%4f'), ...
162     ', p-value = ',num2str(pValue,'%4f')]);
163 xlabel('t');
164 ylabel('Frequency');
165 legend('show');

```

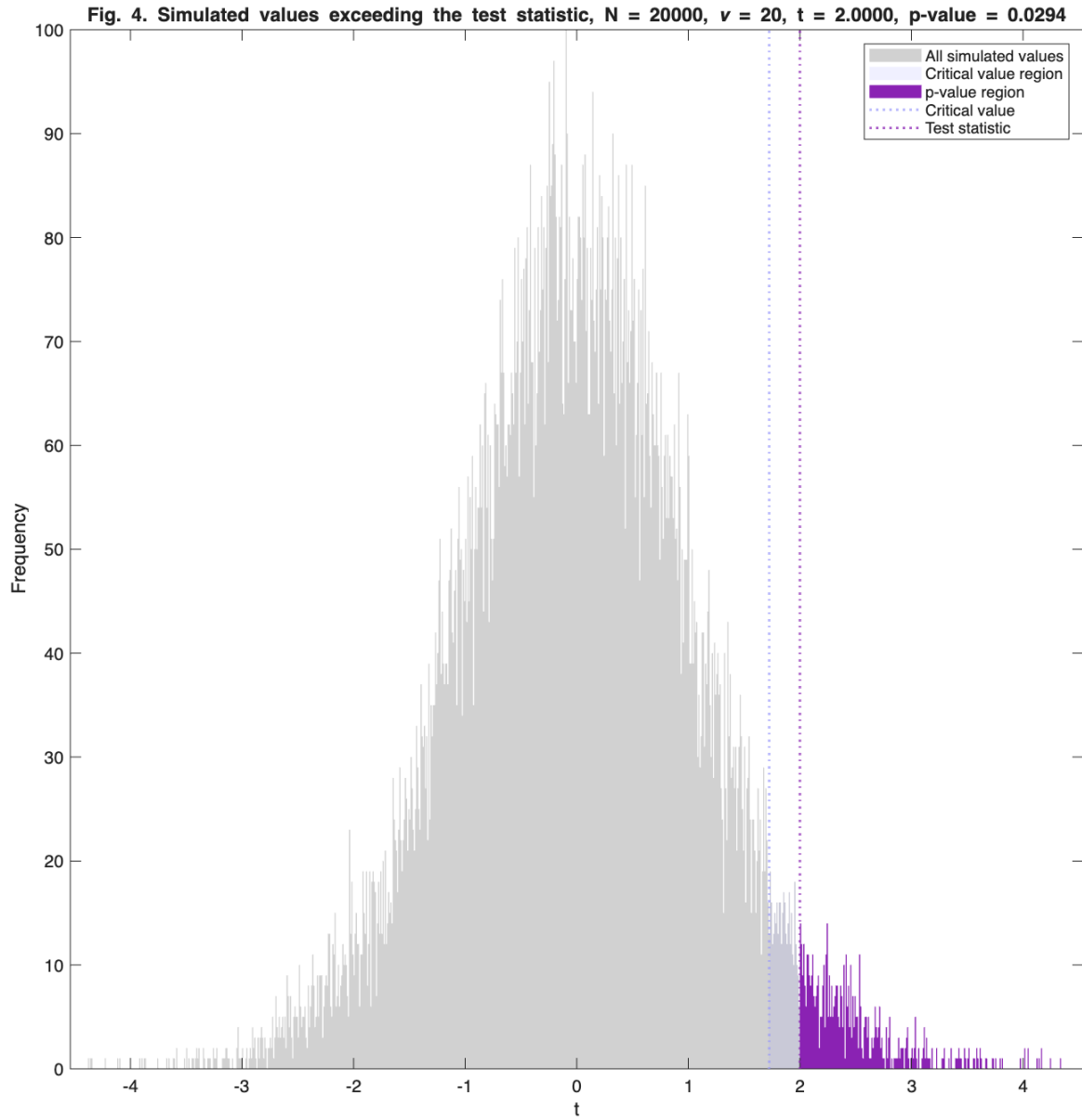


Figure 4: Approximation to the null distribution with the rejection region and p -value region highlighted

7. Theoretical p -value calculation

In the previous section, we approximated the p -value by simulation. In practice, we usually calculate the p -value directly from the limiting distribution of the test statistic under the null hypothesis rather than approximating it by simulation. Let $X \sim t_{20}$ denote a random variable following a t -distribution with 20 degrees of freedom. Since the observed test statistic is $t = 2$, the corresponding one-sided p -value is

$$P(X \geq 2).$$

To calculate this probability, we use the cumulative distribution function, defined as

$$F_X(x) = P(X \leq x).$$

Since

$$P(X \geq 2) = 1 - F_X(2),$$

the p -value can be computed in MATLAB using the built-in `cdf` function. In this example, the theoretical p -value is 0.0312, which is very close to the simulated p -value obtained in the previous section. The small difference arises because the simulated p -value is based on a finite number of random draws, whereas the theoretical p -value is calculated directly from the limiting distribution.

```
167 %% 7. Theoretical p-value calculation
168
169 % 7.1. Define evaluation points
170 xValues = -6:0.1:6;
171
172 % 7.2. Evaluate the PDF of the t-distribution
173 pdfValues = PDF(xValues,nu);
174
175 % 7.3. Plot the PDF and shade the p-value area
176 figure;
177 set(gcf,'Position',[100 100 1000 1000]);
178 plot(xValues,pdfValues,...
179      'Color',[0.000 0.000 0.000],...
180      'DisplayName','PDF');
181 hold on
182 % Critical value
183 xline(criticalValue,...
184      'LineStyle',':',...
185      'Color',[0.604 0.604 0.988],...
186      'LineWidth',1.4,...
187      'DisplayName','Critical value');
188 % Alpha = 0.05 rejection region
189 alphaRange = criticalValue:0.001:6;
190 alphaDensity = pdf('T',alphaRange,nu);
191 alphaArea = area(alphaRange,alphaDensity,...
192      'DisplayName','Critical value region');
193 alphaArea.FaceColor = [0.604 0.604 0.988];
194 alphaArea.FaceAlpha = 0.15;
```

```

195 alphaArea.EdgeColor = 'none';
196 % Test statistic
197 xline(tValue,...
198     'LineStyle',':',...
199     'Color',[0.541 0.133 0.702],...
200     'LineWidth',1.4,...
201     'DisplayName','Test statistic');
202 % p-value region
203 tailRange = tValue:0.001:6;
204 tailDensity = pdf('T',tailRange,nu);
205 tailArea = area(tailRange,tailDensity,...
206     'DisplayName','p-value region');
207 tailArea.FaceColor = [0.541 0.133 0.702];
208 tailArea.FaceAlpha = 1.00;
209 tailArea.EdgeColor = 'none';
210 xmin = min(-criticalValue,max(-10,icdf('T',0.0001,nu)));
211 xmax = max(criticalValue,min(10,icdf('T',0.9999,nu)));
212 xlim([xmin xmax]);
213 hold off
214 title(['Fig. 5. Theoretical PDF of the t-distribution with shaded',...
215     ' p-value area', ...
216     ', \nu = ',num2str(nu)]);
217 xlabel('t');
218 ylabel('Density');
219 legend('show');
220
221 % 7.4. Compare the theoretical and simulated p-values
222 theoreticalPValue = 1 - cdf('T',tValue,nu);

```

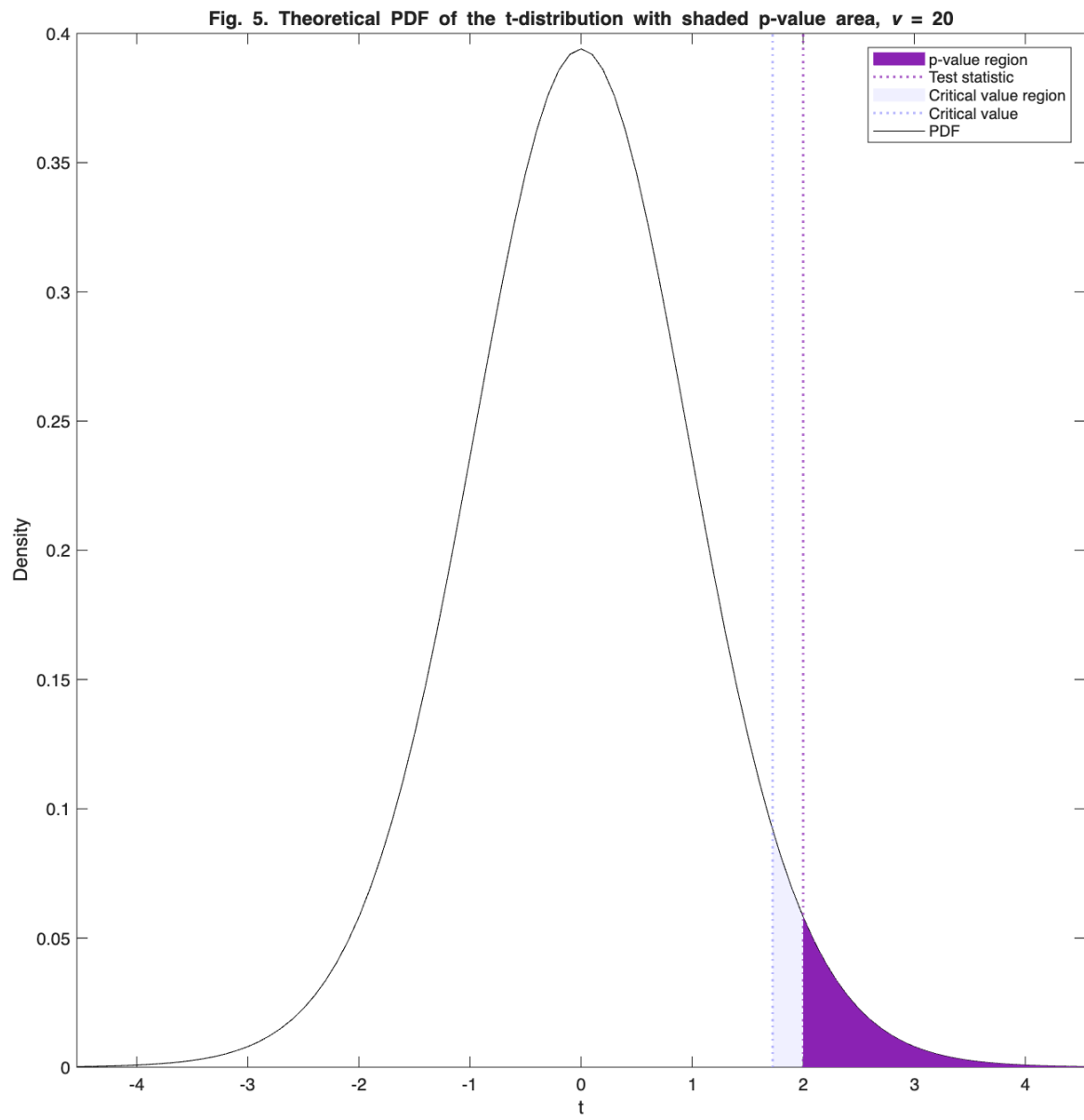


Figure 5: Limiting distribution with the rejection region and p -value region shaded

8. Final notes

This file is prepared and copyrighted by Akash Boelens,, Renata Istrătescu, and Tunga Kantarcı. This file and the accompanying MATLAB files are available on GitHub and can be accessed via this [link](#).